Project Documentation: Dynamic Guardian

1. Project Overview

The **Dynamic Guardian** is a real-time driver and vehicle safety monitoring system built for the **LilyGo T-Display-S3** microcontroller. It leverages an on-device **Edge Impulse Machine Learning model** to analyze simulated sensor data, classifying driver state and vehicle behavior into four categories: **fatigue, stress, road condition (pothole), and braking style**.

The system provides three distinct outputs:

1. **On-device:** A live LVGL (Light and Versatile Graphics Library) dashboard displaying the risk levels.
2. **Developer-side:** A detailed summary printed to the Serial Monitor every 5 seconds.
3. **Cloud-side:** Processed data is pushed to a **Firebase Realtime Database** for remote monitoring or data logging.

2. Core Functionality

- **Real-time ML Inference:** Uses a pre-trained Edge Impulse model (`run_classifier`) to process 19 distinct features.
- **Data Windowing:** Collects 10 samples of data (one every 500ms) into a 5-second "window" before running inference.
- **Data Pre-processing:** Averages the 19 features over the 5-second window and then normalizes them using hardcoded `scaler_mean` and `scaler_scale` values before inference.
- **Heuristic Logic:** Applies post-inference adjustments (e.g., manually increasing the fatigue score if CO2 levels are high) to improve result accuracy.
- **Rich GUI Dashboard:** Displays fatigue and stress on animated bars, shows pothole and braking predictions on a live line chart, and provides simple text-based status updates.
- **IoT Cloud Integration:** Connects to WiFi and uploads a JSON payload with the latest status (e.g., "Driver drowsy," "Road: Poor Condition") to Firebase every 5 seconds.
- **Data Simulation: Crucially, this code does not read from real sensors.** It uses a function (`generate_live_reading`) to create realistic *dummy data* with occasional "event spikes" to simulate driver events.

3. System Architecture & Data Flow

The code operates on a continuous loop, processing data in a clear, multi-stage pipeline:

1. **Input (Simulation):**
    - Every **500ms**, `generate_live_reading()` creates a new set of 19 "sensor" values.
    - This sample is added to a 10-slot circular buffer (`window_buffer`). This buffer always holds the most recent 5 seconds of data (10 samples * 500ms).
2. **Process (Inference & Analysis):**
    - This part runs continuously in the main `loop()`.
    - **Average:** It first calculates the *average* for each of the 19 features across the entire 10-sample window.
    - **Scale:** These 19 *averages* are then normalized using the `scale_feature()` function. This step is **mandatory** for the ML model to work correctly.
    - **Infer:** The 19 scaled features are passed to the Edge Impulse `run_classifier()` function.
    - **Extract:** The four resulting scores (fatigue, stress, pothole, brake) are extracted from the `result` object.
    - **Adjust:** Heuristic rules are applied to fine-tune the ML model's output (e.g., `if (avg[0] > 95) stress += 0.2;`).
3. **Output (Reporting):**
    - **Every 5 Seconds (Serial & Firebase):**
        - Converts the numeric scores (e.g., `0.9`) into human-readable strings (e.g., "High").
        - Generates an "Overall" status (e.g., "Driver drowsy — recommend rest.").
        - Prints a full report to the **Serial Monitor**.
        - Packs these strings and key averages into a `FirebaseJson` object and uploads it to the `/guardianReport/latest` path in your **Firebase Database**.
    - **Every 10 Seconds (Display):**
        - Calls `update_dashboard()` to refresh the **LVGL screen**.
        - Updates the bars, percentage labels, chart, and status text with the latest scores.

4. Key Code Components Explained

`setup()`

- Initializes the Serial Monitor for debugging.
- Connects the device to the specified **WiFi SSID**.
- Initializes the AMOLED display and the **LVGL library**.
- Calls `ui_init()` to draw the dashboard elements for the first time.
- Configures and authenticates with the **Firebase Realtime Database**.
- "**Warms up**" the `window_buffer` by filling it with 10 initial dummy readings.

`loop()`

This is the main scheduler. It has three primary tasks:

1. **Data Collection:** Checks if 500ms have passed since the last sample. If so, it gets a new reading and updates the `window_buffer`.
2. **Inference & Reporting:** Calls `run_inference_and_handle()` on *every single loop* to process the data and handle the 5-second and 10-second report timing.
3. **UI Management:** Calls `lv_task_handler()` on every loop. This is essential for LVGL to handle animations, respond to (potential) touch events, and re-draw the screen.

`generate_live_reading(float *r)`

- This is the **data simulator**. It populates the 19-feature array `r` with random values.
- The values are based on the `scaler_mean` and `scaler_scale` arrays, ensuring the data looks "normal" to the ML model.
- It includes several `if (randf(0,1) > 0.9)` blocks to randomly inject "**event spikes**" (e.g., a sudden high heart rate or high G-force) to test the model's response.
- **In a real-world application, you would replace the body of this function with actual sensor-reading code (e.g., `analogRead()`, I2C/SPI communication).**

`run_inference_and_handle()`

This is the "brain" of the application.

1. **Averages** the `window_buffer`.
2. **Scales** the averaged data.
3. **Calls** `run_classifier()` with the scaled data.
4. **Checks for** an `EI_IMPULSE_ERROR`.
5. **Applies** heuristic rules to the results.
6. **Handles the 5-second timer (`SERIAL_UPDATE_INTERVAL`):**
    - Prints the detailed Serial report.
    - Builds the `FirebaseJson` object.
    - Calls `Firebase.RTDB.setJSON()` to upload the data.
    - Measures and prints the Firebase upload latency.
7. **Handles the 10-second timer (`DISPLAY_UPDATE_INTERVAL`):**
    - Calls `update_dashboard()` to refresh the screen.

`ui_init()` & `update_dashboard(...)`

- `ui_init()`: This function runs once at setup. It creates all the LVGL objects (labels, bars, charts) and places them at their specific coordinates on the screen.
- `update_dashboard(...)`: This function is called every 10 seconds to update the *values* of the objects created in `ui_init()`. It sets the bar percentages, updates the text labels, and adds the next data point to the line chart.

5. Dependencies

This project requires the following libraries:

- `LilyGo_AMOLED.h`: Hardware-specific driver for the T-Display-S3 AMOLED screen.
- `LV_Helper.h`: A helper library (likely provided with the board) to simplify LVGL setup.

- `Edge_device_inferencing.h`: Your specific header file exported from Edge Impulse, containing the ML model and classification functions.
- `WiFi.h`: Standard ESP32 library for WiFi connectivity.
- `Firebase_ESP_Client.h`: Library for connecting to and interacting with Firebase services.